Colonies de fourmis et problème du voyageur de commerce

Une approche expérimentale : la simulation confirme-t-elle la théorie ?

Ivano Balboni Quentin Wache

Novembre 2026

Plan

1. Le problème : pourquoi le TSP est difficile
2. Le modèle ACO et ses variantes
3. Ce que prédit la théorie
4. Protocole expérimental
5. Résultats : complexité, convergence, paramètres
6. Comparaison avec les algorithmes classiques
7. Applications et limites

Objectif : ne pas seulement « faire marcher » l'ACO, mais confronter chaque prédiction théorique à une mesure.

Le problème

Le voyageur de commerce (TSP)

- n villes, distances d_{ij}
- Trouver le cycle le plus court qui passe une fois par chaque ville
- Problème NP-difficile : aucun algorithme polynomial connu
- Nombre de tournées distinctes : $\dfrac{(n-1)!}{2}$

n	tournées possibles
10	$1{,}8 \times 10^{5}$
20	$6{,}1 \times 10^{16}$
30	$4{,}4 \times 10^{30}$
50	$3{,}0 \times 10^{62}$
100	$4{,}7 \times 10^{155}$

À un milliard de tournées par seconde, $n = 20$ prend déjà 2 ans ; $n = 30$ dépasse l'âge de l'Univers d'un facteur 10^4.

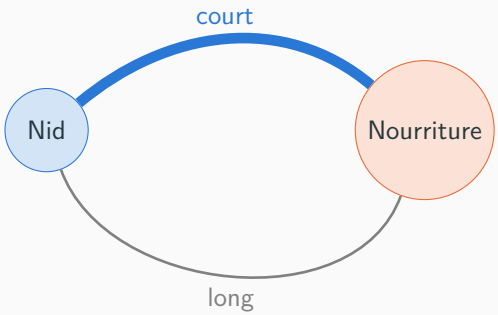
Méthodes exactes et heuristiques classiques

Méthode	Type	Complexité	Garantie
Force brute	exacte	$O(n!)$	optimum
Held-Karp (prog. dynamique)	exacte	$O(n^2 2^n)$	optimum
Plus proche voisin	glouton	$O(n^2)$	aucune
2-opt	recherche locale	$O(n^2)$ par passe	optimum local
Christofides	approximation	$O(n^3)$	$\leq 1{,}5\times$ optimum
Recuit simulé	métaheuristique	paramétrable	asymptotique

- Mesuré : Held-Karp résout 12 villes en 0,03 s, mais la mémoire $O(n\,2^n)$ bloque vers $n \approx 20$
- Au-delà : il faut accepter une solution approchée $\Rightarrow$ métaheuristiques, dont l'ACO

Le modèle ACO

Inspiration : l'expérience du double pont



- Les fourmis déposent une phéromone sur leur chemin
- Sur le chemin court, les allers-retours sont plus rapides ⇒ plus de dépôt
- Plus de phéromone ⇒ plus de fourmis : rétroaction positive
- L'évaporation efface les mauvais chemins

Deneubourg et al. (1990) ; communication indirecte par l'environnement = *stigmergie*

Ant System (Dorigo et al., 1996)

À chaque itération, m fourmis construisent chacune une tournée.

1. Choix de la ville suivante (fourmi k en i, villes non visitées $\mathcal{N}_k$) :

$$p_{ij} = \frac{\tau_{ij}^{\alpha}\, \eta_{ij}^{\beta}}{\sum_{l \in \mathcal{N}_k} \tau_{il}^{\alpha}\, \eta_{il}^{\beta}} \qquad \eta_{ij} = \frac{1}{d_{ij}}$$

2. Mise à jour des phéromones (après toutes les fourmis) :

$$\tau_{ij} \leftarrow \underbrace{(1-\rho)\,\tau_{ij}}_{\text{évaporation}} + \underbrace{\sum_k \frac{Q}{L_k} \mathbb{1}[(i,j) \in T_k]}_{\text{dépôt}}$$

α : poids de la mémoire collective

β : poids de la distance (glouton)

ρ : vitesse d'oubli

m : nombre de fourmis, L_k : longueur de la tournée k

Variantes implémentées

AS Ant System : toutes les fourmis déposent

MMAS MAX-MIN Ant System (Stützle & Hoos, 2000) : seule la meilleure fourmi dépose, $\tau \in [\tau_{\min}, \tau_{\max}]$ avec $\tau_{\max} = \frac{1}{\rho L^*}$, réinitialisation en cas de stagnation

ACS Ant Colony System (Dorigo & Gambardella, 1997) : avec probabilité $q_0 = 0{,}9$, choix de la meilleure arête ; mise à jour locale pendant la construction

+ 2-opt Recherche locale appliquée aux meilleures tournées de chaque itération

Implémentation
Toutes les fourmis avancent en parallèle (opérations sur des matrices $m \times n$, PyTorch).
Le même code tourne sur CPU et sur GPU.

Ce que prédit la théorie

Complexité d'une itération

- Construction : m fourmis $\times$ $(n-1)$ pas $\times$ $O(n)$ candidats

$$\Rightarrow O(m\,n^2)$$

- Évaporation $O(n^2)$, dépôt $O(m\,n)$
- Total pour T itérations : $\boxed{O(T\,m\,n^2)}$
- Mémoire : matrice des phéromones $O(n^2)$

Prédictions testables

- n fixé : temps $\propto m$ (pente 1 en log-log)
- $m = n$: temps $\propto n^3$ (pente 3 en log-log)
- GPU : les m fourmis en parallèle, mais les n pas restent séquentiels

Dynamique des phéromones et convergence

Évaporation seule : $\tau_t = \tau_0(1-\rho)^t$

$\Rightarrow$ la colonie « oublie » en environ $1/\rho$ itérations $\quad (\rho = 0{,}2 : 5 \text{ it.} ; \rho = 0{,}02 : 50 \text{ it.})$

Dépôt constant Δ : $\tau_t \to \Delta/\rho$ (point fixe)

Résultats de convergence

- Gutjahr (2000) : un Ant System modifié trouve l'optimum avec une probabilité qui tend vers 1
- Stützle & Dorigo (2002) : avec $\tau_{\min} > 0$, la probabilité de trouver l'optimum tend vers 1

Mais

Garanties asymptotiques uniquement : aucun temps polynomial (le TSP est NP-difficile). Ce qui compte en pratique : la vitesse de convergence et le risque de stagnation.

Protocole expérimental

Protocole

Instances

- 4 aléatoires uniformes dans $[0, 100]^2$: $n = 12, 50, 100, 200$
- 5 TSPLIB à optimum prouvé : eil51, berlin52, st70, kroA100, ch150

Mesures

- Écart à l'optimum (%) ; si inconnu, au meilleur trouvé
- Temps de calcul, itération de convergence
- Facteur de branchement (diversité des phéromones)

Répétitions

- Algorithmes aléatoires : 5 graines (10 pour l'étude des paramètres)
- Résultats : moyenne $\pm$ écart-type

Budgets

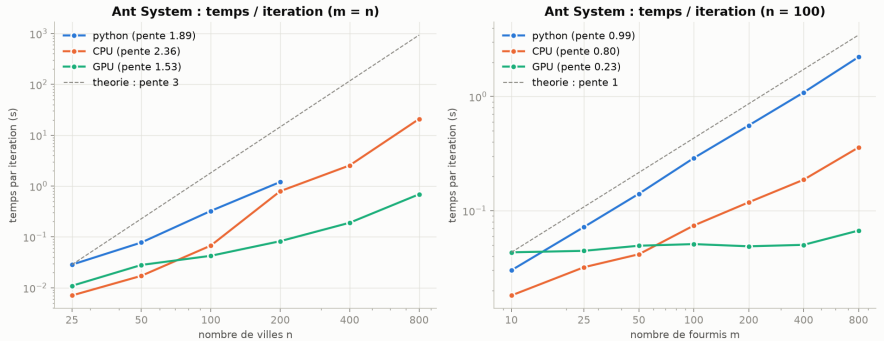
- Variantes ACO : 200 itérations, $m = n$ fourmis
- AS d'origine : 20 fourmis $\times$ 100 itérations

Matériel

- Intel i7-8750H (6 cœurs), GTX 1050 Ti 4 Go

Résultats

Complexité mesurée vs théorie



Pente log-log	Python	CPU vectorisé	GPU
temps vs m (théorie 1)	0,99	0,80	0,23
temps vs n, $m = n$ (théorie 3)	1,89	2,36	1,53

Pourquoi les pentes s'écartent de la théorie

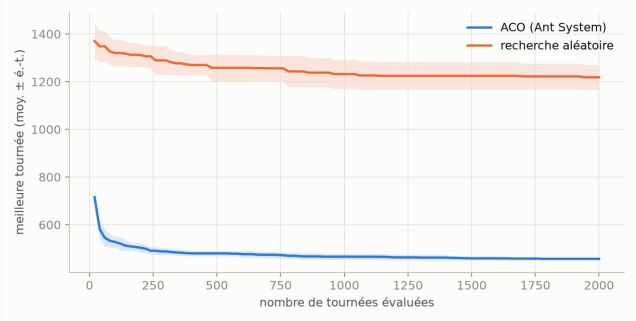
- m **variable :** chaque fourmi ajoute le même travail $\Rightarrow$ pente 1 en Python, comme prévu
- n **variable, pente $<$ 3 :** la boucle sur les $O(n)$ candidats est une seule opération vectorisée. Pour $n \leq 400$, son coût fixe domine son coût proportionnel à n : on mesure $\approx m \times n$ pas, soit une pente proche de 2
- **GPU :** les m fourmis sont traitées en même temps $\Rightarrow$ temps presque constant en m (pente 0,23) tant que la carte n'est pas saturée

Conséquence pratique

$n = m$	50	200	800
CPU (ms / itération)	17	793	20 846
GPU (ms / itération)	28	82	689

Le GPU est plus lent sous ≈ 100 villes, puis jusqu'à 30 $\times$ plus rapide.

L'ACO apprend : comparaison avec le hasard

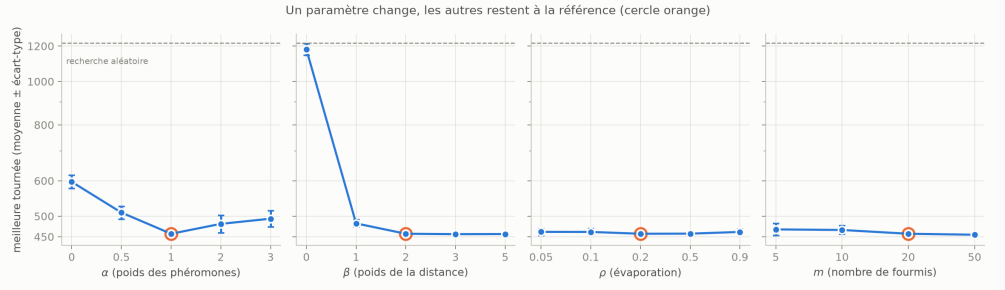


30 villes, même budget : 2 000 tournées

- Recherche aléatoire : 1218 ± 52
- ACO : $457 \pm 4{,}5$

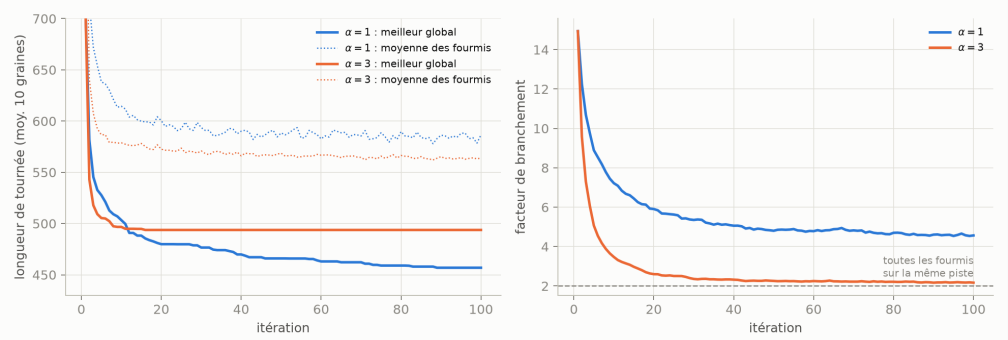
Les tirages aléatoires sont indépendants : ils ne couvrent qu'une fraction négligeable des $4{,}4 \times 10^{30}$ tournées. L'ACO accumule de l'information dans les phéromones.

Influence des paramètres (30 villes, 10 graines)



- $\beta = 0$ (distance ignorée) : presque aussi mauvais que le hasard
- $\alpha = 0$ (pas de mémoire) : 597 ; α trop grand : convergence prématurée
- ρ : peu d'effet ici ; plus de fourmis : résultats plus réguliers (é.-t. 14,6 $\rightarrow$ 1,1)

Convergence prématurée ($\alpha = 3$)



Avec $\alpha = 3$, la colonie converge en ≈ 10 itérations puis ne progresse plus (494 contre 457). Le facteur de branchement tombe vers 2 : toutes les fourmis suivent la même piste, l'exploration s'arrête.

Comparaison avec les algorithmes classiques

Qualité : écart à l'optimum (%, moyenne des graines)

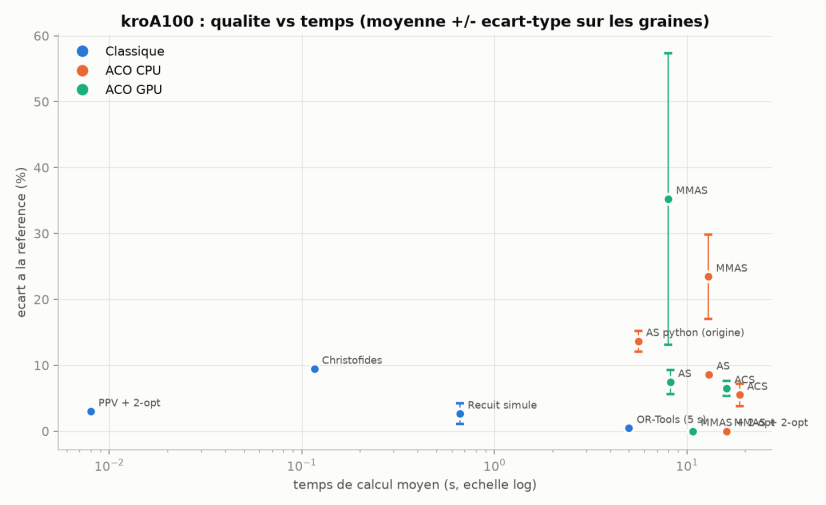
	eil51	berlin52	st70	kroA100	ch150	rand200*	temps kroA100
Plus proche voisin	20,0	19,1	23,0	30,7	25,5	23,2	0,001 s
PPV + 2-opt	2,1	4,0	9,3	3,0	1,4	4,9	0,01 s
Christofides	8,5	13,5	14,2	9,4	10,0	10,5	0,1 s
Recuit simulé	2,2	1,9	1,5	2,6	5,7	6,0	0,7 s
OR-Tools (5 s)	1,2	3,2	1,2	0,5	2,3	3,9	5 s
AS (code d'origine)	8,6	5,3	9,9	13,7	11,6	20,9	5,6 s
AS	6,7	1,5	7,3	7,4	6,0	13,3	8,2 s
ACS	1,2	0,9	2,8	6,5	6,7	14,5	16 s
MMAS	13,2	8,8	19,9	35,2	37,2	47,1	8,0 s
MMAS + 2-opt	0,05	0,0	0,0	0,0	0,24	0,02	11 s

Variantes ACO sur GPU (même qualité sur CPU). * instance aléatoire : écart au meilleur tour trouvé, l'optimum n'est pas connu. Tableau complet (9 instances, CPU et GPU) en annexe.

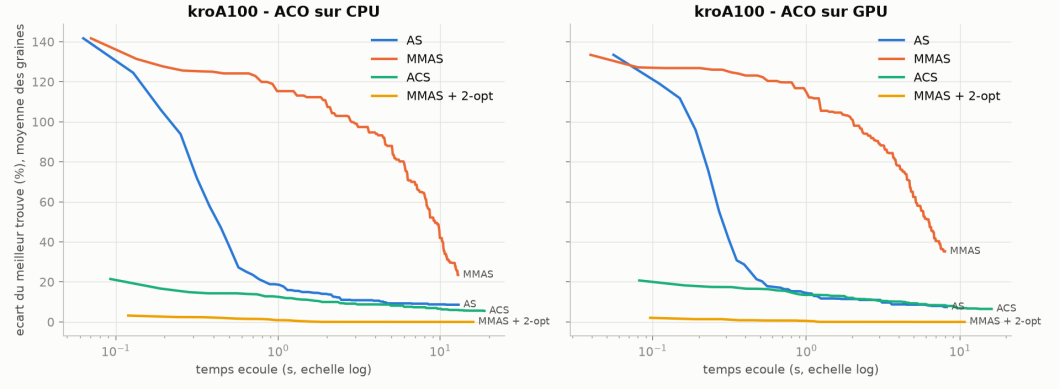
Ce que montre le tableau

- MMAS + 2-opt atteint l'optimum prouvé sur berlin52, st70 et kroA100 à chaque graine ; +0,05 % sur eil51, +0,24 % sur ch150
- Il fait mieux que **OR-Tools** limité à 5 s (+0,5 à +3,9 %), mais en 4 à 90 s
- Les ACO sans recherche locale restent à 5–15 % : plus proche voisin + 2-opt fait mieux en 0,02 s
- Le code d'origine du dépôt est le moins bon des ACO (budget plus petit : 2 000 tournées)
- Held-Karp confirme l'optimum sur 12 villes : tous les algorithmes l'atteignent

Qualité vs temps de calcul (kroA100)



Convergence des variantes (kroA100)

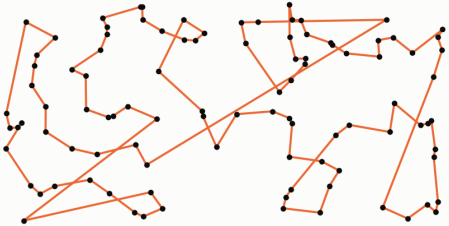


MMAS seul semble mauvais, mais sa courbe descend encore : avec $\rho = 0{,}02$, la mémoire dure $\approx 1/\rho = 50$ itérations, conformément à la théorie il lui faut bien plus que 200 itérations. La recherche locale accélère fortement la convergence.

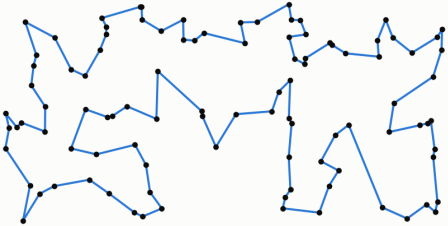
Le résultat visuellement

kroA100 (optimum prouvé : 21 282)

Plus proche voisin : 27807 (+30.7 %)



MMAS + 2-opt : 21282 (+0.0 %)



Applications et limites

Où l'ACO est utile

Problèmes de graphes difficiles

- Tournées de véhicules (livraisons, collecte)
- Ordonnancement d'ateliers
- Affectation quadratique
- Sélection de variables en apprentissage

Environnements dynamiques

- Routage dans les réseaux (AntNet, Di Caro & Dorigo, 1998) : les phéromones s'adaptent quand les coûts changent

Où ce n'est pas utile

Plus court chemin sur un graphe fixe : Dijkstra est exact en $O((V+E)\log V)$. Une métaheuristique n'a d'intérêt que si le problème est NP-difficile ou change dans le temps.

Atouts

Parallélisable (GPU), facile à adapter à de nouvelles contraintes, fournit plusieurs bonnes solutions.

Limites de l'ACO et de notre étude

De l'algorithme

- Aucune garantie de qualité en temps fini
- Sensible aux paramètres (α, β, ρ)
- Coût $O(m\,n^2)$ par itération
- Seul, moins bon que des heuristiques simples : la recherche locale est indispensable

De notre étude

- Budgets non identiques entre algorithmes : comparer sur le graphique qualité/temps
- Instances ≤ 200 villes
- Paramètres de MMAS prévus pour des milliers d'itérations
- Une seule machine

Conclusion

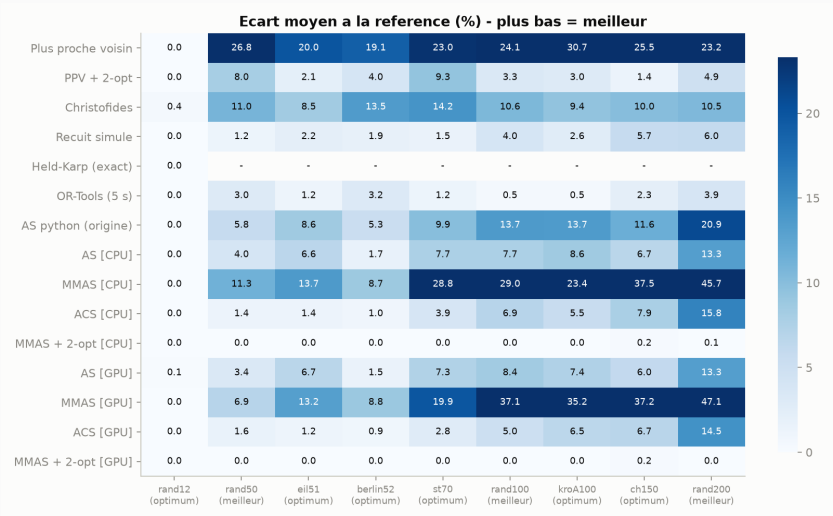
1. **La théorie se vérifie, avec nuances :** temps linéaire en m (pente 0,99) ; en n, les coûts fixes de l'implémentation masquent le n^2 aux tailles testées
2. **La dynamique prévue s'observe :** apprentissage par rétroaction (2,7 $\times$ mieux que le hasard), stagnation quand α est trop grand, mémoire en $1/\rho$
3. **L'ACO seul ne suffit pas :** c'est combiné à une recherche locale qu'il atteint l'optimum, au prix d'un temps de calcul que le GPU réduit (jusqu'à 30 $\times$)

Merci ! Questions ?

Références

- M. Dorigo, V. Maniezzo, A. Colorni. *Ant System : optimization by a colony of cooperating agents*. IEEE Trans. SMC-B, 26(1), 1996.
- M. Dorigo, L. M. Gambardella. *Ant Colony System*. IEEE Trans. Evol. Comp., 1(1), 1997.
- T. Stützle, H. Hoos. *MAX-MIN Ant System*. Future Gen. Comp. Systems, 16(8), 2000.
- W. J. Gutjahr. *A graph-based Ant System and its convergence*. Future Gen. Comp. Systems, 16(8), 2000.
- T. Stützle, M. Dorigo. *A short convergence proof for a class of ACO algorithms*. IEEE Trans. Evol. Comp., 6(4), 2002.
- J.-L. Deneubourg, S. Aron, S. Goss, J. M. Pasteels. *The self-organizing exploratory pattern of the Argentine ant*. J. Insect Behavior, 3, 1990.
- G. Di Caro, M. Dorigo. *AntNet : distributed stigmergetic control for communications networks*. JAIR, 9, 1998.
- M. Dorigo, T. Stützle. *Ant Colony Optimization*. MIT Press, 2004.
- G. Reinelt. *TSPLIB*. ORSA J. Computing, 3(4), 1991.

Annexe : tableau complet des écarts (%)



Annexe : reproduire les résultats

Code : `github.com/BalboniIvanoWacheQuentin/using-ACO-for-TSP`

- `python experiments.py` : étude des paramètres (`results/runs.csv`, `history.csv`)
- `python compare.py` : comparaison des 15 algorithmes sur 9 instances
- `python scaling.py` : temps par itération (complexité)
- `python plot_compare.py` et `slides/make_figures.py` : figures
- `streamlit run streamlit.py` : démonstration interactive